

MiCU: 基于大语言模型的端到端智能家居指令理解

韩浩伟^{*†}武汉大学计算机
学院 中国武汉
haowei.han@whu.edu.cn

胡可欣[†]小米公司 中
国武汉
h
ukexin@xiaomi.com

蔡伟伟 小米公司 中国
武汉
c
aiweiwei@xiaomi.com

张德彪 小米公司 中国武汉
zhangdebiao@xiaomi.com

秦斌 小米公司 中
国武汉
qi
nbin@xiaomi.com

王宇翔 武汉大学 计算机
学院 中国武汉
n
ai.yxwang@whu.edu.cn

蒋佳伟[‡]武汉大学 计算机
学院 中国武汉
jiawei.jiang@whu.edu.cn

严潇 武汉大学 数学与人工
智能研究院 中国武汉
ya
nxiaosunny@whu.edu.cn

杜博[‡]武汉大学 计算
机学院 中国武汉
dubo@whu.edu.cn

摘要

智能家居生态系统中的指令理解系统可以自动化设备控制并显著提升用户体验。然而，尽管它们在处理精确表述（例如，“打开卧室灯光”）时表现良好，但在处理模糊或意图不明确的指令（例如，“让卧室变得舒适”）时却面临困难。大语言模型（LLMs）在各个领域具有良好的泛化能力，在此类任务上可以超越传统的基于规则的系统，但其有效性常常受限于稀缺的领域特定数据、不足的任务特定适配以及高昂的计算成本。在本文中，我们提出了一种利用用户日志和大语言模型的自动化训练数据合成 workflow；随后我们构建了 MiCU，一个擅长指令理解的领域特定大语言模型。具体而言，我们采用课程学习将领域知识注入基础大语言模型，然后通过结合领域特定思维规则引导的强化学习（RL）进行冷启动训练，以增强其推理能力。此外，我们引入了一种令牌压缩技术，将设备描述压缩为单个特殊令牌，从而显著降低推理开销，并催生了 MiCU-fast——一个针对长输入优化的高效变体。大量实验表明，MiCU 显著优于基线模型，在所有设备类别上平均准确率增益达到 20.01%。我们已在小米智能家居应用中部署了 MiCU，每日接收约 170 万次页面浏览量。生产环境评估显示，

^{*}在小米公司实习期间完成的工作。

[†]两位作者对本研究的贡献相同。

[‡]通讯作者。



本作品采用知识共享署名 4.0 国际许可协议进行许可。KDD 2026, 韩国济州岛。

© 2026 版权所有/作者所有。ACM ISBN 979-8-4007-2259-2/2026/08

<https://doi.org/10.1145/3770855.3818446>

MiCU 将用户纠正率降低了 1.57%，并将人工审核准确率提高了 32.05%。我们的数据和代码可在 https://github.com/xiaomi-research/iot_spec_llm 获取。

CCS 概念

• 计算方法学 → 信息提取；• 人本计算 → 普适与移动计算。

关键词

智能家居，指令理解，大语言模型

ACM 参考文献格式：

韩浩伟、胡可欣、蔡伟伟、张德彪、覃彬、王宇翔、蒋佳伟、严潇、杜博。2026。MiCU：基于大语言模型的端到端智能家居指令理解。载于第 32 届 ACM SIGKDD 知识发现与数据挖掘会议论文集 V.2 (KDD 2026)，2026 年 8 月 9 日至 13 日，韩国济州岛。ACM，美国纽约州纽约市，共 11 页。<https://doi.org/10.1145/3770855.3818446>

1 引言

智能家居生态系统已从根本上改变了日常生活，提供了极大的便利，并成为现代生活的重要组成部分。小米智能家庭平台作为中国领先的智能家居生态系统，已开发出超过 113 个子类别的终端设备，例如灯光、窗帘和空调，覆盖了日常生活的诸多方面。小米智能家居通过物联网技术连接这些设备，并通过其移动应用（即米家应用）进行管理。截至 2025 年 9 月，小米智能家居生态系统实现了一个重要里程碑，其连接的物联网设备数量突破了 10 亿大关。这一增长得到了一个拥有 1.146 亿月活跃用户的强大社区的支持，其中包括超过 2160 万拥有五台或更多智能设备的用户。¹

¹小米公司，2025 年第三季度业绩公告

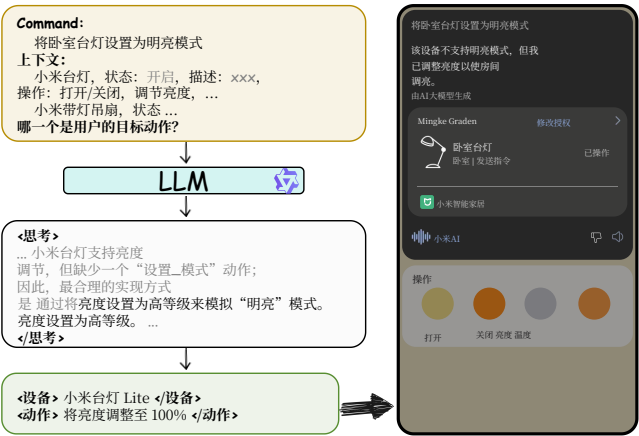


图1: 小米智能家居生态系统中基于LLM的端到端指令理解流程示意图。

通过自然语言进行设备控制, 使用户能够通过直观的语音指令来管理小米智能家居生态系统。鉴于设备的巨大多样性及其复杂的操作, 自然语言交互相比僵化的手动控制 (例如遥控器或应用菜单) 提供了更优越的灵活性, 显著减少了用户在多设备管理上的精力。然而, 实现这一能力具有挑战性, 因为许多用户指令是模糊或不匹配的。例如, 用户可能说“将卧室台灯设置为明亮模式”, 而实际上意图是让台灯更亮, 而不是选择一个“明亮”模式 (该台灯并不支持此模式)。在此类场景中, 系统必须分析指令, 考虑上下文, 并推断用户的预期动作。传统方法, 例如基于规则的选择 [34], 通常采用多阶段启发式流程, 通过手动设计的规则 (例如设备名称匹配和动作名称匹配) 将指令映射到潜在动作, 这增加了系统复杂性, 并限制了泛化能力和准确率。

大语言模型 (LLMs) 通过海量训练语料库获得了广泛的世界知识和强大的推理能力, 已在包括语义解析 [2, 19]、开放域问答 [9, 31], 和指令遵循 [15, 40] 在内的多个领域展现出良好的泛化能力。通过用户指令和上下文来提示LLMs, 为指令理解提供了一种直观的端到端方法。该策略利用LLMs的推理能力, 有效应对智能家居交互的复杂性。如图1所示, LLMs利用其强大的推理能力, 识别出用户意图是调高亮度, 正确避免了选择不存在的“亮度”模式, 相较于传统方法展现出显著优势。

挑战。如表1所示, 虽然基于LLM的方法相较于传统方法展现出更优越的理解能力, 但直接在智能家居任务中使用通用大语言模型表现有限。我们确定了开发一个有效的基于LLM的端到端指令理解系统的三个关键挑战:

(1) **领域特定数据稀缺。**大语言模型的能力源于海量数据。尽管智能家居场景中的原始用户日志数量庞大, 但由于其非结构化特性, 难以直接用于训练

表1: 基于规则选择 (RS)、DeepSeek-R1和我们的MiCU在所有设备类别上理解智能家居指令的性能对比。

指标	RS	DeepSeek-R1	MiCU
准确率	66.32	73.10	94.61
参数量	N.A.	671B	4B

且缺乏可作为训练监督的真实标签目标动作。

(2) **特定任务适应能力有限。** 尽管大语言模型拥有丰富的世界知识, 但它们缺乏专门的领域专业知识, 因此无法很好地利用上下文信息、推断用户意图并预测目标动作。然而, 专门针对智能家居指令理解任务来适配大语言模型的有效策略尚未得到充分开发。

(3) **计算开销高。** 提示大语言模型会因其庞大的参数量而产生巨大的计算开销和时间延迟。这一挑战在复杂的指令理解任务中进一步加剧, 因为该任务需要包含详细描述设备状态及其他信息 (如可用动作) 的长上下文提示。

为解决**挑战 (1)**, 我们引入了DevCmd, 这是一个基于用户日志合成的难度分级数据集。具体来说, 我们提出了一个自动化合成工作流, 用于生成简单和困难的指令数据。对于简单数据, 该工作流使用定义设备功能能力和约束条件的设备规格来填充预定义模板, 按照结构化格式生成样本及其对应的目标动作作为标签。对于困难数据, 该工作流对困难的真实用户日志进行采样和格式化, 然后利用大语言模型结合检索增强生成 (RAG) [12] 和 SelfRefine [16] 来推断用户意图, 并将目标动作标注为标签。最终得到的数据集包含28个类别的5万个样本, 有效缓解了智能家居指令理解中训练数据稀缺的问题。

随后, 我们构建了用于指令理解的MiCU大语言模型, 以应对剩余的挑战。针对**挑战 (2)**, 我们没有依赖会产生显著上下文开销的手动提示工程, 而是提出了一个有效的训练框架, 使基础大语言模型适应指令理解任务。我们首先通过课程学习 [28], 将专业知识注入基础大语言模型, 训练从简单样本开始, 逐步过渡到困难样本。此外, 我们引导DeepSeek-R1 [5] 生成思维链过程来解决指令理解任务, 并利用这些思维链过程对MiCU进行监督微调。最后, 我们应用强化学习来进一步优化和增强模型的推理能力。针对**挑战 (3)**, 我们发现大量冗余源于用于描述候选设备 (如可用动作) 的冗长提示词。因此, 我们采用一种令牌压缩技术, 将这些冗长的描述压缩成一个特殊令牌, 从而显著降低了计算成本, 并催生了性能依然保持竞争力的MiCU-fast。

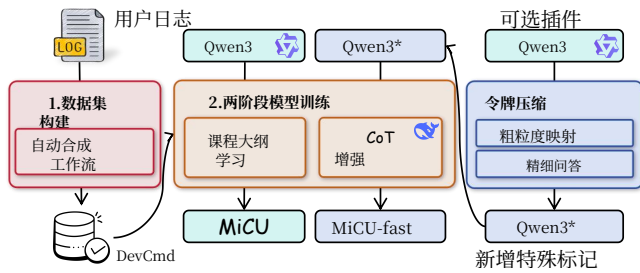


图2: MiCU关键技术的概览。

我们在包含多个设备类别的数据集上评估了MiCU，并将其与基线模型进行了比较，这些基线包括传统的基于规则的选择方法以及采用少样本上下文学习的流行通用预训练大语言模型。结果表明，MiCU在所有设备类别上都优于所有基线模型：与基于规则的方法和性能最佳的大语言模型相比，其准确率分别提升了28.29%和20.01%。消融实验证实了我们训练框架的每个阶段都是有效的。效率研究表明，令牌压缩使提示词长度减少了31.9%，而准确率仅轻微下降了0.6%。在线实验显示，MiCU实现了用户纠正率降低1.57%，人工审核准确率提升32.05%。

本文的贡献总结如下：

- 我们引入了DevCmd，这是一个包含5万个训练样本的数据集，由真实用户日志合成，旨在解决智能家居指令理解领域的数据稀缺问题。
- 我们提出了MiCU，一个专用于智能家居指令理解的大语言模型。它集成了课程学习、思维链监督微调和强化学习，以注入领域知识并增强推理能力。
- 我们设计了一种令牌压缩技术，通过将冗长的设备描述压缩为单个特殊标记，显著降低了计算成本和延迟。
- 在多种设备类别下进行的离线和在线广泛实验，证明了所提出的数据集合成工作流和MiCU的有效性。

2 MiCU用于指令理解

图2描绘了我们构建智能家居指令理解大语言模型MiCU的概览。我们首先通过一个自动化工作流合成领域特定的训练数据集DevCmd，然后使用两阶段训练策略来适配一个基础大语言模型（即Qwen3-4B-instruct [30]）。此外，我们提出了一种可选的令牌压缩方法来降低计算成本，从而产生一个快速变体MiCU-fast。以下章节将详细描述每个组件。

2.1 数据集构建

为缓解智能家居指令训练数据的数据稀缺问题，我们从小米智能家居平台收集了大量用户日志。此数据收集过程在用户明确知情同意下进行，并严格遵守隐私和安全协议，以确保所有敏感信息均已匿名化。然而，这些原始数据本质上是非结构化的，其特点是

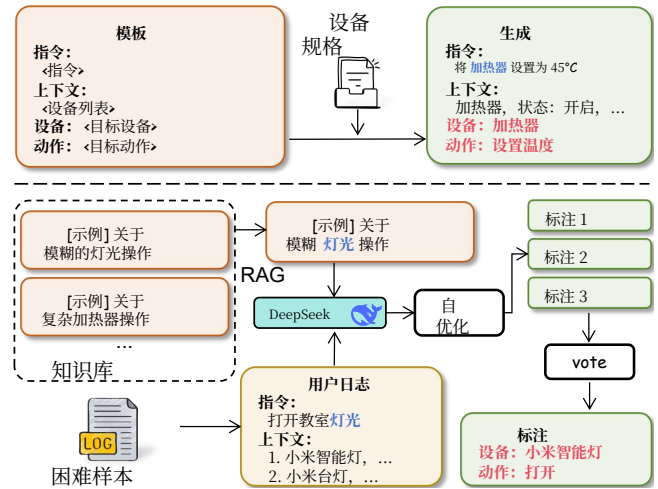


图3: 数据集合成工作流。顶部展示了针对简单数据的直接基于模板的生成，而底部展示了使用DeepSeek针对困难数据的高级标注。

不同设备版本间存在不一致的元数据和异构结构。更重要的是，日志通常包含噪声，因为真实世界场景固有的模糊性使得难以识别真实的用户意图，从而为提取可靠的模型监督真实标签带来了复杂性。

合成工作流。为了构建清晰、格式化的训练数据并带有可靠的标签，我们提出了一种自动化数据合成工作流。该过程利用大语言模型，基于真实用户日志自动生成训练样本。如图3所示，为了实现更高的合成数据质量和更好地适应下游训练，我们采用不同的生成策略来创建“简单”和“困难”的数据样本。这种双策略方法产生了DevCmd D，这是一个专门为训练大语言模型执行智能家居指令理解任务而量身定制的难度分级数据集。如图3中的生成输出所示，D中的每个样本都包含一条用户指令、上下文以及真实标签目标设备和动作。

为了生成简单样本 D^{easy} domain，我们采用了一种基于模板的合成方法，旨在生成遵循所需逻辑和目标结构化输出格式的样本。我们系统性地用每一类设备规格填充精心预定义的模板，从而为每个有效动作生成一个对应的简单样本，同时确保功能完整性。这一策略使我们能够高效地自动收集大量样本，并确保它们天生就拥有真实标签。这种全面的覆盖有助于对下游模型进行稳健的初始训练。

为了生成困难样本 D^{hard} domain，我们直接采用复杂的用户日志，将其格式化，并自动为其标注高质量的标签。①为此，我们首先手动标注少量样本，采用思维链风格思考过程，构建了一个包含近 1K 条目的知识库。然后，我们利用 RAG [12] 方法来自动标注其余样本。具体而言，我们将每个样本作为查询，检索相关的手动标注样本作为少样本示例，并提示 DeepSeek-R1 [5] 以思维链的方式推断

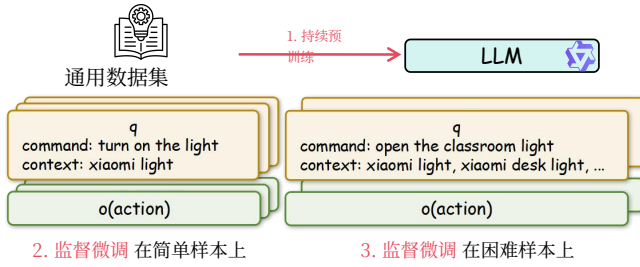


图4: 课程学习阶段。该阶段始于在通用数据集（例如悟道）上对基础大语言模型进行持续预训练（CPT），随后在合成的从易到难的样本上进行渐进式监督微调（SFT）。

目标动作作为标签。② 为了进一步确保合成标签的质量，我们采用 SelfRefine [16] 方法，提示模型反思其思维链标注过程并尝试重新生成。③ 我们收集通过反思生成的多个结果，并利用模型对它们进行投票，将共识结果作为真实标签。最后，我们手动审查并修正由大语言模型标注的标签，确保其可靠性。这种稳健的多阶段方法有效利用了大语言模型的推理能力，在确保生成高质量标签的同时，最大限度地降低了手动标注成本。

数据集详情。在训练方面，我们的数据集 DevCmd 包含 3 万个简单样本和 2 万个困难样本，总计 5 万个样本，覆盖 28 个类别的智能设备，主要是照明设备，但也包括窗帘和空调（AC）。在评估方面，该数据集包含来自用户日志的 9 千个人工标注样本，以确保标签质量并实现更精确的模型评估。关于设备类别的详细信息见附录A。

2.2 课程学习

尽管通用大语言模型在各种任务上表现出色，但由于缺乏针对性知识，它们通常在专业领域表现不佳。在智能家居指令理解中，任务难度差异显著，从简单的单一设备控制到模糊的多意图指令不等。直接在具有如此高难度方差的数据集上训练模型可能导致不稳定和性能下降 [28]。为了解决这个问题，我们采用了课程学习方法，如图4所示，逐步引入知识。我们首先在通用数据上进行持续预训练（CPT），然后在简单样本上进行监督微调（SFT）以建立稳定基础，最后在挑战性困难样本上进行监督微调（SFT），以确保能够稳健地处理复杂查询。

通用持续预训练。为了增强基础模型的通用语言理解和理解能力，我们在进行领域特定学习之前，先在通用数据集上执行持续预训练。训练目标是最小化负对数似然（NLL），持续预训练损失正式定义如下：

$$\mathcal{L}_{\text{CPT}}(\mathcal{D}; \theta) = -\frac{1}{|\mathcal{D}|} \sum_{x \in \mathcal{D}} \sum_{t=1}^{|x|} \log P(x^t | x^{<t}; \theta), \quad (1)$$

其中 $\mathcal{D}$ 代表训练数据集， θ 表示模型参数， $x \in \mathcal{D}$ 是单个样本， x^t 是序列中的第 t 个令牌，而 $x^{<t}$ 表示序列中所有先前的令牌。具体来说，我们首先在两个开源中文语料库——悟道 [37] 和 Firefly [32]，上训练模型，以学习通用的语言和上下文表示，然后在 LongAlign [1] 上对其进行训练，以增强其长上下文理解能力，因为智能家居上下文通常较长。该过程可以表示为：

$$\theta_{\text{CPT}} \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{CPT}}(\mathcal{D}_{\text{general}}; \theta_{\text{base}}). \quad (2)$$

领域特定监督微调。我们在一个持续预训练的模型上进行监督微调，以提升其领域特定能力。在监督微调过程中，模型以用户指令和上下文信息作为输入，并被训练生成相应的目标动作作为输出。形式上，损失函数定义为：

$$\mathcal{L}_{\text{SFT}}(\mathcal{D}; \theta) = -\frac{1}{|\mathcal{D}|} \sum_{(q,o) \in \mathcal{D}} \sum_{t=1}^{|o|} \log P(o^t | q, o^{<t}; \theta), \quad (3)$$

其中 q 表示输入提示，它同时包含指令和上下文信息，而 o 代表模型的输出，此处对应于目标动作 a 。然而，简单与困难样本之间的高难度方差和严重不平衡阻碍了稳定训练 [29]。具体而言，困难样本至关重要但复杂且往往稀缺。在掌握简单样本之前就对困难样本进行训练，会因缺乏坚实基础而导致模型不稳定。虽然混合两种类型样本是可行的，但这会降低性能，因为模型常常对丰富的简单模式过拟合，而无法从稀缺的困难样本中学习。

为了解决这些挑战，我们应用渐进式监督微调。我们首先在 DevCmd 的简单样本上对模型进行微调，以建立涵盖所有有效设备动作的基础知识。这一步使模型能够全面理解基础智能家居指令和设备行为。随后，我们在 DevCmd 的困难样本上对模型进行微调，这些样本包含诸如多重或模糊意图的复杂指令，以及像误导性设备别名这样的挑战性上下文。第二阶段使模型能够有效处理复杂场景，并显著增强其指令理解能力。该微调过程可表述如下：

$$\theta'_{\text{SFT}} \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{SFT}}(\mathcal{D}_{\text{domain}}^{\text{easy}}; \theta_{\text{CPT}}), \quad (4)$$

$$\theta_{\text{SFT}} \leftarrow \arg \min_{\theta} \mathcal{L}_{\text{SFT}}(\mathcal{D}_{\text{domain}}^{\text{hard}}; \theta'_{\text{SFT}}). \quad (5)$$

2.3 思维链增强

大语言模型（LLMs）已展现出通过利用其推理能力生成改进输出的能力。近期的研究主要集中在通过思维链（CoT）方法来运用这种能力，该方法鼓励模型进行逐步推理。然而，让大语言模型自主决定其推理过程会忽略领域特定知识的使用，导致在专业任务中的准确率降低。为了解决这一局限，我们提出了一种领域引导提示，明确指导我们的模型使其推理过程与领域特定规则对齐。具体而言，如图5所示，我们首先使用领域引导提示来指导一个教师模型，

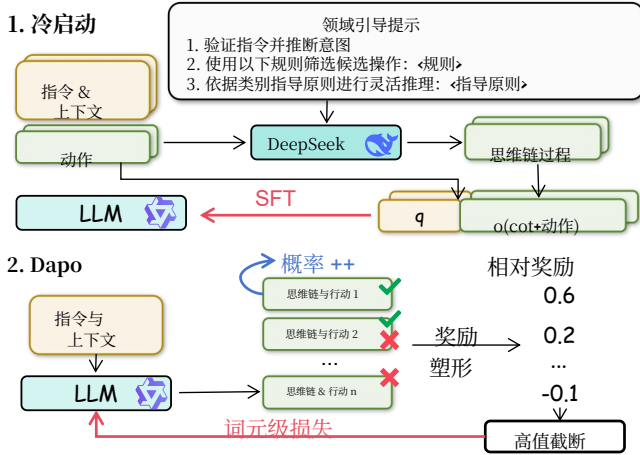


图5: 思维链增强阶段。我们首先通过在领域引导的DeepSeek生成的思维链过程上进行监督微调来冷启动模型，然后使用Dapo强化学习对其进行精炼。

DeepSeek-R1 [21], 为每个样本生成推理过程, 记为 *think*。我们向教师模型提供目标动作, 以实现更高的推理精度。然后, 我们使用生成的推理过程 *think* 作为冷启动, 对我们的模型进行监督微调 (SFT), 并利用强化学习 (RL) 进一步增强其推理能力, 最终提升模型的性能。

领域引导提示。如图5所示, 由领域引导提示引导的推理过程包含三个步骤。首先, 模型验证用户指令的有效性并分析其意图, 例如识别用户想要操作哪个设备类别。其次, 模型应用基于规则的过滤过程来缩小潜在操作的范围。此过程涉及匹配设备名称、识别房间名称, 并采用与预定义规则一致的其他过滤机制。通过将大语言模型的高级推理能力与精心设计的业务规则相结合, 此步骤比传统方法实现了更高的准确率。最后, 如果前几步未能确定目标动作, 则指示模型使用特定类别的指导方针灵活地对指令进行推理。通过采用这种领域引导提示, 大规模教师模型大语言模型可以生成更一致、更准确的思维链推理过程, 从而显著增强我们模型的训练。

思维链冷启动。我们通过监督微调, 以思维链推理能力对模型进行冷启动。具体而言, 将指令及其对应的上下文 (不含领域引导提示) 作为输入, 通过监督微调训练模型, 使其模仿预先生成的推理过程 *think* 并预测目标动作 *a*。基于前一训练阶段, 我们将此阶段整合到课程学习中, 其中包含简单与困难样本。为简单样本生成了总计17K个思维链推理过程, 而困难样本则为8K个。监督微调训练损失定义于公式(3)中, 此处输出 *o* 不仅包含目标动作 *a*, 还包含思考过程 *think* (即 $o = \{think, a\}$)。我们将包含思维链思考过程的数据集记为 $\mathcal{D}_{CoT}$ 。

该过程可形式化表达为:

$$\theta_{CoT} \leftarrow \arg \min_{\theta} \mathcal{L}_{SFT}(\mathcal{D}_{CoT}; \theta_{SFT}). \quad (6)$$

使用DAPO进行强化学习。在通过思维链冷启动训练模型, 使其具备遵循引导过程的强大推理基础后, 我们进一步利用 DAPO [33], (一种基于GRPO [21]构建的大规模强化学习策略) 通过强化学习来增强其推理能力。简而言之, DAPO的核心思想是让模型为同一个输入提示生成多个输出, 计算每个输出的相对优势, 并奖励格式良好且准确的输出以提高其概率。将模型作为策略进行优化的目标如下:

$$\mathcal{J}_{DAPO}(\mathcal{D}; \theta) = \mathbb{E}_{(q, a) \sim \mathcal{D}, \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(\cdot | q)} \quad (7)$$

$$\left[\frac{1}{\sum_{i=1}^G |o_i|} \sum_{i=1}^G \sum_{t=1}^{|o_i|} \min(r_i^t(\theta) \hat{A}_i^t, \text{clip}(r_i^t(\theta), 1 - \epsilon_l, 1 + \epsilon_h) \hat{A}_i^t) \right]$$

s.t. $0 < |\{o_i | \text{is_equivalent}(a, o_i)\}| < G,$

其中 G 表示采样输出的数量, ϵ_l 和 ϵ_h 分别是截断的下界和上界, π_{θ} 和 π_{old} 分别表示当前策略和旧策略, $r_i^t(\theta)$ 代表在当前策略下输出 o_i^t 的词元级重要性权重, 而 $\hat{A}_i^t$ 表示归一化的优势函数:

$$r_{i,t}(\theta) = \frac{\pi_{\theta}(o_i^t | q, o_i^{<t})}{\pi_{old}(o_i^t | q, o_i^{<t})}, \quad \hat{A}_{i,t} = \frac{R_i - \text{mean}(\{R_i\}_{i=1}^G)}{\text{std}(\{R_i\}_{i=1}^G)}. \quad (8)$$

与GRPO相比, DAPO采用了高值截断、动态采样、词元级损失和过长奖励塑形等技巧来改进强化学习过程, 从而提供更稳定的训练和更高的准确率。

我们设计的奖励 R_i 包含三个组成部分: 响应标签检查、格式验证检查和内容正确性检查, 以确保模型在内容和格式上都能产生正确答案。响应标签检查鼓励模型在指定标签之间进行正确推理和预测, 这些标签包括 `<think>` 和 `</think>`、`<device>` 和 `</device>`、`<action>` 和 `</action>`。格式验证检查验证模型的结构化输出 (JSON) 在语法上是否有效, 并符合预期的字段和类型。内容正确性检查评估预测的正确性, 包括设备、动作类型和相应的值。这些组件的权重可根据需要进行调整, 以平衡格式严格性和理解准确率。总体而言, 我们可以将强化学习过程正式表述如下:

$$\theta_{MiCU} \leftarrow \arg \max_{\theta} \mathcal{J}_{DAPO}(\mathcal{D}_{domain}; \theta_{CoT}). \quad (9)$$

2.4 令牌压缩

尽管MiCU经过专业数据训练, 能有效处理智能指令理解任务, 但它会产生巨大的计算开销和延迟。我们的分析显示, 平均上下文提示包含353个令牌, 远超平均52个令牌的输出。值得注意的是, 详细描述设备特性和允许操作的设备描述, 占上下文提示总长度的94%。这些描述带来了巨大的计算开销, 且随着用户设备数量的增加, 这一负担呈线性增长。然而, 省略此类描述会损害模型学习全面设备



图6：令牌压缩技术。该模型经历一个由粗到精的适应过程，从描述-特殊令牌对的双向映射，逐步进展到针对特定任务的问答微调。

信息的能力，并导致理解能力下降。因此，我们应用令牌压缩技术来压缩这些描述，并加速我们的MiCU，从而产生了其高效变体 MiCU-fast。

特殊令牌设计。设备类别的描述作为详细规格，使得能够使用令牌压缩技术将每个冗长的类别规格表示为一个唯一的令牌 t_{spec} 。为此，我们提出一种策略，将冗长的规格转换为紧凑的令牌。如图6所示，对于每个设备类别，所有可用动作及其对应的值范围被压缩成一个单一的 t_{spec} 。例如，一个列举了多种功能能力（例如，调节亮度）和有效参数约束（例如，0-100）的冗长文本描述，被一个代表性的令牌所取代，例如 `<spec_token_1>`。在此压缩之后，一个设备仅由其 t_{spec} 及其简化的状态来表示，该状态表征了所有操作属性的当前状态。这种用紧凑令牌替代详尽规格的做法，显著减少了令牌总数，从而在降低计算开销和推理延迟的同时，提高了编码效率。

适应训练。为确保模型能有效理解并利用这些特殊标记，我们提出了一种如图6所示的由粗到精的训练方法进行适应。首先，我们在一个双向映射任务上训练模型：给定一个特殊令牌 t_{spec} 作为输入，模型被训练生成其对应的规格。反之，给定一个类别规格，模型被训练预测关联的 t_{spec} 。这种双向映射使模型能快速适应新添加的令牌，确保对令牌化规格的鲁棒理解。接下来，我们在一个更细粒度的任务上训练模型：我们向模型查询当前令牌 t_{spec} 的详细信息，模型被训练生成正确的动作名称及其有效值范围。这个细粒度任务确保模型透彻理解每个特殊标记，从而实现准确高效的下游命令理解。值得注意的是，此压缩阶段被整合在两阶段模型训练之前，以便模型在充分令牌压缩的数据上进行训练，从而增强其对下游任务的适应性和有效性。

3 实验评估

在本节中，我们进行了广泛的实验，以解决以下研究问题：

- 研究问题一：我们的模型 MiCU 在数据集 DevCmd 上相比基线方法取得了怎样的总体准确率？
- 研究问题二：在训练框架中，课程学习阶段和思维链增强阶段的效果如何？
- 研究问题三：令牌压缩技术能在多大程度上高效提升我们的模型 MiCU 的性能？
- 研究问题四：我们需要更多还是更少的训练数据/模型参数？
- 研究问题五：我们的模型 MiCU 在在线部署中的表现如何？

3.1 实验设置

数据集。为进行细粒度评估，我们根据目标设备类别将评估集划分为多个子集。我们为两项任务选取并展示最常见且最重要的设备类别。训练和评估统计数据在第2.1节中描述，更多细节在附录A中提供。

任务。我们报告了每种方法在评估数据上针对两项侧重点不同的任务的准确率性能：

- **设备识别 (DevIdent)。**此任务评估，给定一条指令和一个简单的候选设备列表，模型能否识别用户的目标设备（不包括动作）。
- **动作预测 (ActPred)。**此任务评估，给定一条指令和包含其完整描述（包括可用动作）的候选设备，模型能否预测用户的预期动作及关联值（例如，“设置模式”和“亮度”）。

基线模型。我们将MiCU和MiCU-fast与三种基线类型进行比较：1)

传统基线：基于规则的选择 (RS) 采用基于规则的工作流，通过匹配设备名称、设备属性、房间名称等属性来筛选目标动作。2) **专有大语言模型。**我们评估了强大的专有大语言模型，包括GPT-4o-mini、GPT-4o [7]、DeepSeek-R1 [5]、和DeepSeek-V3 [14]。3) **开源大语言模型。**我们还评估了流行的开源大语言模型，包括Llama [4]和我们的基础模型Qwen3 [30]。值得注意的是，所有基于大语言模型的基线都提供了少样本示例和详细指南，以确保公平比较。其他专用方法因不适用于此复杂任务而被排除在外。

3.2 主要结果 (研究问题一)

表2报告了在评估数据上，传统基线模型、基于大语言模型的基线模型、MiCU及其高效变体MiCU-fast之间的准确率比较。我们有以下三点观察结果：

MiCU和MiCU-fast在DevIdent和动作预测任务中均持续优于所有基线模型。通过利用大语言模型的卓越推理能力，MiCU相比RS实现了28.29%的总体增益。尽管仅有40亿参数，MiCU超越了参数量大得多的基于大语言模型的基线模型，相比DeepSeek-V3提升了20.01%，相比Qwen3-30B提升了30.00%，这得益于其在我们训练框架内有效学习了任务特定知识并融入了领域引导的思维链推理。

在我们的模型中，虽然MiCU-fast在DevIdent和总体任务上略逊于MiCU，但它在动作预测方面表现出色。这是因为动作预测提供了完整设备描述——这正是我们压缩的主要目标；在此，特殊标记有效地替代了冗长描述。相反，对于DevIdent，简化描述导致MiCU-fast中的特殊标记引入了任务无关冗余。我们将在第3.4节进一步分析这种权衡。

表2：指令理解评估。报告了准确率(%)。最佳结果以**粗体**标出。“灯光”表示此子数据集中所有目标动作的设备均为灯光。增益表示相对于最佳性能基线的改进。

方法	设备识别					动作预测					总体
	灯光	AC	Fan	窗帘	整体	灯光	AC	交换机	套接字	整体	
基于规则的选择	61.37	72.52	74.19	71.92	76.98	73.00	30.00	57.02	60.87	45.53	66.32
专有大语言模型											
GPT-4o-mini	33.97	52.01	51.55	34.13	71.04	45.37	55.49	31.40	50.31	49.66	63.79
GPT-4o	66.99	57.51	62.37	40.08	80.43	50.79	67.84	41.32	62.11	60.19	73.57
DeepSeek-R1	72.58	60.17	65.09	46.67	79.51	49.89	68.04	36.36	50.93	60.60	73.10
DeepSeek-V3.2	76.65	77.66	74.23	51.19	82.46	48.76	65.88	30.74	37.27	59.29	74.60
开源大语言模型											
Llama3.1-8B	41.95	38.97	39.66	23.17	63.65	36.79	37.84	34.13	27.33	36.09	54.31
Llama3.3-70B	45.99	43.25	45.18	29.90	70.03	40.18	49.91	38.55	36.08	45.18	61.61
Qwen3-4B	31.19	10.60	28.88	13.65	39.19	29.57	17.84	28.93	36.65	24.21	34.11
Qwen3-30B	51.52	42.18	49.25	29.99	72.85	42.25	51.80	38.55	39.50	46.79	64.01
MiCU-4B	89.81	94.51	92.27	93.25	96.80	87.03	92.01	83.24	80.49	90.33	94.61
MiCU-4B-fast	88.22	94.51	91.75	94.44	95.89	87.21	90.78	88.43	85.09	90.35	94.01
Gain	13.16	16.85	18.04	43.25	14.34	36.42	23.97	47.11	22.98	29.75	20.01

表3：课程学习的消融研究。报告了总体准确率（%），最佳值已加粗。难易顺序先使用简单样本训练模型，再使用困难样本。混合在训练期间结合简单与困难样本。

Idx	方法	DevIdent	动作预测	总体
1	仅简单样本	48.84	20.79	39.31
2	仅困难样本	94.92	<u>88.61</u>	92.78
3	Easy+Hard (混合)	96.32	86.56	<u>93.01</u>
4	Hard→Easy	68.36	62.49	66.37
5	Easy→Hard (本文方法)	96.80	90.33	94.61

在两项任务中，模型在DevIdent上的表现均优于动作预测，因为后者需要预测精确的动作及对应值，因此更具挑战性。对于设备类别子集，“灯光”类别的准确率通常较低，因为经常存在多个相似的候选设备使识别复杂化，且灯光支持多种有效操作，这扩大了动作空间并增加了混淆。

3.3 消融研究 (研究问题二)

为了研究训练阶段中每个组件的影响，我们分别对它们进行了全面的消融研究。
课程学习。如表3所示，我们比较了仅简单样本、仅困难样本、混合样本集、从难到易的进度安排以及我们提出的从易到难的课程。关键观察结果：仅困难样本（2）的表现优于仅简单样本（1），并取得了第二好的动作预测准确率，这表明困难样本包含更丰富的信息，尤其对于复杂的动作预测。在两种类型的样本上进行训练（3）比单独使用任一类型都获得了更好的DevIdent和总体结果，证明了它们的互补性。从难到易的

表4：思维链增强的消融研究。报告了总体准确率（%），最佳值已加粗。

方法	DevIdent	动作预测	总体
基线 (Qwen3-4B)	39.19	24.21	34.11
+冷启动	79.86	67.00	75.50
+RL	87.64	80.04	85.06
基线+课程学习	95.73	89.68	93.68
+冷启动	96.57	88.83	93.95
+强化学习 (本文方法)	96.80	90.33	94.61

进度安排（4）表现比混合设置（3）更差，这表明训练顺序很重要。因此，我们的从易到难课程（5）在两项任务上都取得了最佳结果。

思维链增强。如表4所示，我们在两种思维链增强设置下评估了模型在使用和不使用课程学习时的性能：冷启动，以及冷启动后接强化学习。我们观察到以下几点。首先，课程学习为思维链增强提供了坚实基础，并持续提升准确率。其次，思维链冷启动通过训练模型以领域引导的思维链风格进行推理，带来了显著增益，使其比直接预测答案更能有效理解复杂案例。第三，强化学习进一步优化了模型，增强了其推理能力，带来了额外的性能提升。

3.4 效率研究 (研究问题三)

图7展示了应用我们的令牌压缩技术前后令牌数量和准确率的变化情况。关键观察结果如下。首先，准确率与令牌数量呈负相关，

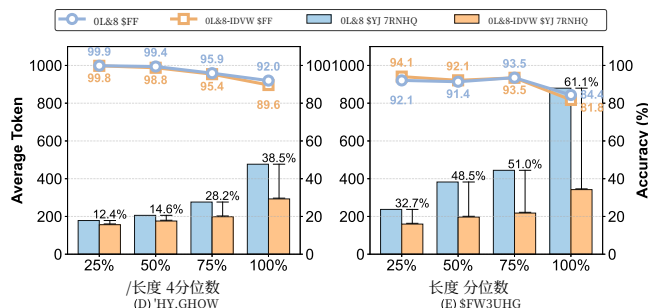


图7：效率研究。图中展示了MiCU和MiCU-fast的平均令牌数量及相应的准确率。

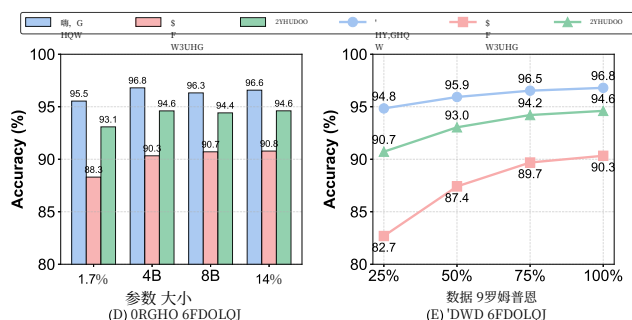


图8：可扩展性研究。图(a)展示了不同基础模型（Qwen3）规模下的性能，而图(b)展示了使用Qwen3-4B时不同训练数据规模下的性能。

因为更长的提示会引入更多候选设备和消歧复杂性。因此，仅需通过简化描述进行设备识别的DevIdent任务，比动作预测任务产生的令牌更少且准确率更高。其次，MiCU-fast展示了可扩展的效率，实现了12.4%到61.1%之间的压缩率。虽然极端压缩可能导致轻微的准确率损失，但MiCU-fast在48.5%的压缩率下显著提高了动作预测任务的准确率。通过将详细描述压缩为单个令牌，它过滤了与任务无关的噪声，同时保留了功能语义。这种效果对于动作预测任务中冗余的描述非常显著，但对于设备识别任务中本就极简的输入则效果有限；因此，MiCU-fast在动作预测任务上的表现相对优于设备识别任务。总体而言，我们的技术将令牌开销降低了31.9%，并将每秒查询率提升了13.5%，在效率和性能之间实现了有利的权衡。

3.5 扩展性研究（研究问题四）

我们通过考察参数量和训练数据量的影响进行了扩展性研究，如图8所示。我们评估了不同规模的Qwen3模型作为基础模型。结果表明，非常小的模型（例如1.7B）受限于能力且表现不佳，而将模型规模增加到4B以上（至8B和14B）在动作预测任务中并未带来显著增益，实际上在设备识别任务中反而导致性能下降。在我们有效的训练策略下，一个4B参数的基础模型足以在智能家居指令理解任务中表现出色。我们还评估了

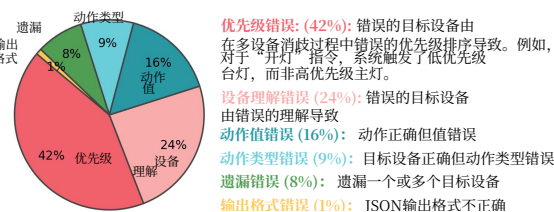


图9：主要类别“灯光”的失败案例分析。

训练数据规模的影响，方法是按比例增加各难度级别和类别的示例。对于设备识别任务，一旦使用了50%–75%的数据，性能几乎达到平台期；扩展到100%仅带来边际增益。对于动作预测任务，性能在75%–100%的范围内保持相对平稳，我们将此归因于该任务复杂度更高，需要更多样本才能完全学习。综上所述，这些结果表明，对于智能家居指令理解任务，我们当前的数据集足以确保训练收敛。

3.6 在线部署（研究问题五）

我们将提出的MiCU部署在小米智能家居的智能家居指令理解系统中，使用了四个RTX4090（24GiB）实例，每个实例配置了40 GiB内存，每天服务约170万页面浏览量。部署的服务现已处理全部的智能家居指令理解流量。在98每秒查询率的持续负载下，该服务保持了173毫秒的平均延迟和263毫秒的P95延迟，满足了智能家居交互的实时性要求。

用户纠正。我们将MiCU与生产线（一个基于规则的系统，该系统使用预定义的启发式规则（例如，设备名称、属性和房间名称）来匹配设备动作）进行了比较。为了量化实时环境中的性能（在实时环境中，真实标签本质上是缺失的），我们采用了用户纠正率作为主要的在线指标。用户纠正率衡量了用户手动纠正系统预测动作的频率，直接反映了意图预测的失败率。在一周的A/B测试中，MiCU被部署来处理50%的生产流量，结果表明MiCU将用户纠正率从4.16%显著降低至2.59%。相较于基线37.37%的相对减少，凸显了我们的模型在真实世界场景中卓越的可靠性和理解能力。

人工审核。为了进一步验证模型在真实世界指令上的性能，我们对从生产日志中抽取的样本案例进行了人工审核。具体而言，我们为每个分支随机抽取了570个复杂会话，并优先处理照明和空调等高频核心类别。我们邀请了专家标注员来验证预测动作是否与用户的潜在意图一致。为确保高质量标注，所有样本均经过独立双标注，任何标注不一致之处均通过二次审核解决。结果表明，MiCU将准确率从47.1%显著提升至78.3%。这项细粒度审核证明，我们的模型能够有效理解用户指令，并解决传统基于启发式规则的系统无法处理的意图模糊性。

失败案例分析。我们抽取了100个灯光类别的失败案例，这是最常见的类别。我们将失败原因分为六类，如图9所示。大多数失败是由设备优先级错误引起的，这表明设备间复杂的优先级关系对模型而言难以学习。格式错误最为少见，表明我们的训练框架能有效帮助40亿参数模型遵循指令。其余的失败主要源于复杂指令和上下文依赖，这对模型的指令理解能力构成了挑战。

4 相关工作

大语言模型在智能家居中的应用。近年来，已有若干工作探索将大语言模型应用于智能家居任务。ChatIoT [3] 支持从自然语言进行零代码触发-动作编程，允许终端用户通过大语言模型创建物联网自动化规则。LLM4TAP [26] 从文本描述中推断用户需求与意图，并将此信号与通过奇异值分解增强的图相结合，置于一个对比学习框架中，以改进TAP规则推荐。SAGE [18] 通过将生成限制在大语言模型控制动作集内并使用动态构建的提示树，将指令理解锚定在具体的智能家居操作上，从而在新的基准测试上相比先前的大语言模型基线取得了显著增益。Sasha [11]通过引导大语言模型经过五个结构化启发式步骤，来处理欠明确的、目标导向指令。尽管取得了这些进展，这些方法主要关注TAP或目标导向推理，而非直接的指令理解。因此，它们不适用于我们的特定任务，而我们的MiCU填补了这一研究空白。

指令理解。指令理解是一个长期存在的挑战。早期的系统 [17] 依赖于手动语法和启发式关键词提取。后续基于BERT的模型 [8, 10, 36] 改进了语义解析，但仍需要启发式方法进行动作映射。大语言模型最近因其强大的上下文推理和泛化能力 [22, 23, 27] 而被用于指令理解。与传统流水线不同，大语言模型擅长消歧、意图推断和多步推理，从而提高了端到端准确率。为了缩小通用大语言模型与领域特定指令理解之间的差距，两种策略很常见：提示工程和后训练。提示工程 [20, 24] 通过精心设计的提示在推理时向模型提供领域知识和推理指导原则，而后训练 [29, 35]（例如，领域特定指令微调）则将领域知识直接注入模型权重。我们采用后训练来提升性能，同时减少冗长提示带来的推理开销，这呼应了在训练和推理加速方面更广泛的努力 [6, 25, 38, 39]。我们进一步引入了自动化数据合成、包含课程学习和思维链增强的两阶段训练，以及一种令牌压缩技术。这些创新共同赋予MiCU在智能家居领域卓越的指令理解能力。

5 结论

我们介绍 MiCU，一个用于智能家居的基于大语言模型的指令理解系统。具体而言，我们使用课程学习来注入

基础知识到基础大语言模型中。接着，我们采用思维链增强来提升模型的推理能力。最后，我们提出一种令牌压缩技术，将冗长的设备描述压缩为单个令牌，从而降低模型的计算成本。大量的离线/在线实验表明，我们的 MiCU 优于所有基线模型，为智能家居指令理解的社区研究铺平了道路。

致谢

我们衷心感谢小米公司的张帆、王思文和牛凯欣在本项目全程中富有建设性的讨论和宝贵的支持。本工作由国家自然科学基金（编号：62225113, 62472327）、新基石科学基金会通过新基石研究员项目、湖北省创新研究群体项目（编号：2024AFA017）以及湖北省重点研发计划（编号：2023BAB077）资助。本工作得到了武汉大学-金山联合实验室的支持。

参考文献

[1] 白雨石、吕欣、张佳杰、何宇泽、齐骥、侯磊、唐杰、董宇骁、李涓子。2024. LongAlign：大语言模型长上下文对齐的配方。发表于计算语言学协会年会：EMNLP 研究结果。[2] Andrew Drozdov, Nathanael Schärli, Ekin Akyürek, Nathan Scales, Xinying Song, Xinyun Chen, Olivier Bousquet, Denny Zhou. 2022. 使用大语言模型进行组式语义解析。arXiv预印本 arXiv:2209.15003 (2022)。[3] 高毅、肖凯杰、李富、徐伟峰、黄嘉明、董伟。2024. ChatIoT：基于触发-动作的物联网程序零代码生成。ACM交互式、移动、可穿戴和普适技术汇刊 (IMWUT) 8, 3, 文章103 (2024)。[4] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan等。2024. Llama 3模型群。arXiv预印本 arXiv:2407.21783 (2024)。[5] 郭大雅、杨德建、张浩伟、宋俊晓、张若愚、徐润鑫、朱启昊、马世荣、王培毅、毕晓等。2025. Deepseek-r1：通过强化学习激励大语言模型的推理能力。arXiv预印本 arXiv:2501.12948 (2025)。[6] 胡闯、鲍巍、王丹、刘凤鸣。2019. 面向边缘推理加速的动态自适应DNN手术。发表于IEEE计算机通信会议 (INFOCOM) 论文集。[7] Aaron Hurst, Adam Lerer, Adam P Goucher, Adam Perelman, Aditya Ramesh, Aidan Clark, AJ Ostrow, Akila Welihinda, Alan Hayes, Alec Radford等。2024. GPT-4o系统卡。arXiv预印本 arXiv:2410.21276 (2024)。[8] 蒋一迪、Bidisha Sharma, Maulik Madhavi、李海洲。2021. 从BERT Transformer到Speech Transformer的知识蒸馏用于意图分类。arXiv预印本 arXiv:2108.02598 (2021)。[9] Ehsan Kamalloo, Nouha Dziri, Charles Clarke, Davood Rafiei. 2023. 评估大语言模型时代的开放域问答。发表于计算语言学协会年会 (ACL) 论文集。[10] Minjeong Kim, Gyuwan Kim, Sang-Woo Lee, Jung-Woo Ha. 2021. St-BERT：用于端到端口语理解的跨模态语言模型预训练。发表于IEEE声学、语音与信号处理国际会议 (ICASSP)。7478–7482。[11] Evan King, Haoxiang Yu, Sangsu Lee, Christine Julien. 2024. Sasha：智能家居中大语言模型的创造性目标导向推理。ACM交互式、移动、可穿戴和普适技术汇刊 (IMWUT) 8, 1, 文章12 (2024)。[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel等。2020. 面向知识密集型NLP任务的检索增强生成。发表于神经信息处理系统进展 (NeurIPS)。[13] 林宇豪、汤志鹏、童佳彦、肖俊清、鲁斌、李宇航、李超、张治国、王俊华、罗浩等。2026. FuxiShuffle：阿里云上用于分布式数据处理的适应性与弹性Shuffle服务。arXiv预印本 arXiv:2602.22580 (2026)。

[14] 刘爱新、梅傲雪、林邦才、薛冰、王炳璇、徐秉政、吴伯超、张博伟、林超凡、董晨等。2025. Deepseek-v3.2: 推进开放大语言模型的前沿。arXiv预印本arXiv:2512.02556 (2025)。[15] 娄人泽、张凯、尹文鹏。2024. 大语言模型指令遵循: 进展与挑战综述。计算语言学50, 3 2024 1053 1095 (), -。[16] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang等。2023. Self-refine: 基于自我反馈的迭代精炼。神经信息处理系统进展 (NeurIPS) (2023)。[17] Mahda Noura, Sebastian Heil, Martin Gaedke。2020. VISH: 您的智能家居对话系统也需要训练数据吗?。载于Web工程, Maria Bielikova, Tommi Mikkonen, Cesare Pautasso (编)。171–187。[18] Dmitriy Rivkin, Francois Hogan, Amal Feriani, Abhisek Konar, Adam Sigal, Steve Liu, Greg Dudek。2023. Sage: 基于接地执行的智能家居代理。arXiv预印本arXiv:2311.00772 (2023)。[19] Subhro Roy, Samuel Thomson, Tongfei Chen, Richard Shin, Adam Pauls, Jason Eisner, Benjamin Van Durme。2023. Benchclamp: 语言模型句法与语义解析评估基准。发表于神经信息处理系统进展 (NeurIPS)。[20] Pranab Sahoo, Ayush Kumar Singh, Sriparna Saha, Vinija Jain, Samrat Mondal, Aman Chadha。2024. 大语言模型中提示工程的系统综述: 技术与应用。arXiv预印本arXiv:2402.07927 (2024)。[21] 邵志宏、王培毅、朱启昊、徐润鑫、宋俊晓、毕晓、张浩伟、张明川、YK Li、吴洋等。2024. Deepseekmath: 在开放语言模型中突破数学推理的极限。arXiv预印本arXiv:2402.03300 (2024)。[22] Tom Silver, Varun Hariprasad, Reece S Shuttlesworth, Nishanth Kumar, Tomás Lozano-Pérez, Leslie Pack Kaelbling。2022. 使用预训练大语言模型进行PDDL规划。发表于NeurIPS 2022基础模型用于决策制定研讨会论文集。[23] Stefanie Tellex, Thomas Kollar, Steven Dickerson, Matthew Walter, Ashis Banerjee, Seth Teller, Nicholas Roy。2011. 理解机器人导航与移动操作的自然语言命令。发表于AAAI人工智能会议 (AAAI) 论文集。[24] 王佳宁、黄文康、邱明辉、石秋慧、王洪斌、李想、高明。2022. 预训练语言模型中的知识提示用于自然语言理解。发表于自然语言处理实证方法会议 (EMNLP) 论文集。[25] 王宇翔、马驰、严潇、黄敏聪、李晓光、韩瑞东、殷彬、陈尚宇、李想、江飞、于磊、刘川、林伟、韩浩伟、周晓凯、杜博、蒋佳伟。2026. MTGenRec: 美团中生成式推荐模型的高效分布式训练系统。发表于ACM SIGKDD知识发现与数据挖掘会议 (KDD) 论文集。[26] 吴刚、胡亮、胡宇霄、熊兴波、王峰。2025. LLM4TAP: 大语言模型增强的TAP规则推荐。IEEE物联网期刊12, 10 2025 13157 13169 (), -。[27] 谢雅琪、陈晨、朱同垚、白锦斌、龚泽、Harold Soh。2023. 使用大语言模型将自然语言翻译为规划目标。arXiv预印本arXiv:2302.05128 (2023)。[28] 徐本峰、张立成、毛振东、王泉、谢洪涛、张永东。2020. 自然语言理解的课程学习。发表于计算语言学会年会 (ACL) 论文集。[29] 徐璨、孙庆峰、郑凯、耿修波、赵璞、冯家展、陶重阳、蒋达新。2023. Wizardlm: 赋能大语言模型遵循复杂指令。发表于国际学习表征会议 (ICLR) 论文集。[30] 杨安、李安峰、杨宝松、张北辰、惠彬源、郑博、于博文、高畅、黄陈根、吕晨轩等。2025. Qwen3技术报告。arXiv预印本arXiv:2505.09388 (2025)。[31] 杨方凯、赵璞、王泽忠、王璐、乔波、张珏、Mohit Garg、林庆伟、Saravan Rajmohan、张冬梅。2023. 赋能大语言模型在工业领域特定问答上表现更佳。发表于自然语言处理实证方法会议 (EMNLP) 论文集。[32] 杨建新。2023. Firefly。https://github.com/yangjianxin1/Firefly。[33] 于启颖、张政、朱若飞、袁宇峰、左晓晨、岳宇、戴维南、范恬恬、刘高红、刘凌君等。2025. Dapo: 一个大规模的开源大语言模型强化学习系统。arXiv预印本arXiv:2503.14476 (2025)。[34] John M. Zelle, Raymond J. Mooney。1993. 使用构造性归纳逻辑编程学习语义语法。发表于AAAI人工智能会议 (AAAI) 论文集。[35] 张晟宇、董林峰、李夏雅、张森、孙晓飞、王书鹤、李继伟、胡润一、张天伟、王贵胤、吴飞。2026. 指令

大语言模型微调: 一项综述。ACM计算概览(2026)。[36] 张卓盛、吴宇威、赵海、李祖超、张帅良、周曦、周翔。2020. 用于语言理解的语义感知BERT。载于AAAI人工智能会议论文集。[37] 赵雪、赵涵宇、袁莎、王业全。2022. 悟道语料库文本。doi:10.57760/sciencedb.o00126.00004[38] 郑浩宇、付方程、吴佳、袁彬航、张永强、王浩、朱媛媛、严潇、蒋佳伟。2026. 基于预测的KV缓存管理实现动态智能体 workflow 高效服务。arXiv预印本arXiv:2605.06472(2026)。[39] 郑浩宇、张永强、付方程、周晓凯、罗浩、朱鸿超、朱媛媛、王浩、严潇、蒋佳伟。2026. 利用不确定性感知的输出长度预测进行大语言模型推理调度。arXiv预印本arXiv:2604.00499(2026)。[40] Jeffrey Zhou, 卢天健、Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, 栾毅、Denny Zhou, 侯乐。2023. 大语言模型的指令遵循评估。arXiv预印本arXiv:2311.07911(2023)。

数据集详情

DevCmd数据集包含28个不同的智能设备类别, 共计5万个样本, 涵盖了现代智能家居应用的全面范围。这些类别包括环境控制设备, 如空调、风扇、加湿器、除湿器、新风系统和电暖器, 以及传感硬件, 如温湿度传感器和空气质量监测器。家庭清洁与维护方面则由扫地机器人、擦地机器人、吸尘器、洗衣机、洗碗机、干衣机和晾衣架代表。此外, 该数据集还纳入了基础设施与照明组件, 包括开关、插座、智能窗帘、开窗器和灯具, 以及中央控制面板。最后, 它涵盖了多样化的娱乐与生活电器, 例如电视、分体电视、电视盒子、机顶盒、投影仪、电饭煲、热水器、浴霸、智能镜子、香薰机、足浴盆和智能花盆。这种广泛的多样性确保了数据集能够捕捉到不同家庭场景下广泛的语言表达和功能指令。

B 实现

我们使用 Qwen3-4B-instruct [30] 作为基础大语言模型, 并使用 deepseek-R1 [5]进行思维链生成。在CPT阶段, 模型以5e-5的学习率训练了1个轮次。对于课程监督微调, 简单阶段和困难阶段分别以5e-6的学习率训练了1个和2个轮次。在思维链增强的强化学习阶段, 我们使用1e-5的学习率, 并将剪裁比率 ϵ_l 和 ϵ_h 分别设置为0.2和0.28以确保稳定性。在所有阶段, 模型都进行了全参数优化 [13]的训练。训练在8块NVIDIA H20 GPU (96GB) 上进行。对于在线部署, 我们采用了FP8量化, 并实现了推测解码, 其前瞻步长为 $k = 4$ 。这种推理策略被证明对我们的任务非常有效, 因为它显著加速了JSON输出中高度结构化令牌的生成。

C 隐私

智能家居指令理解依赖于用户指令和上下文信号 (例如, 设备状态), 这些信息可能包含敏感信息。因此, 我们在数据收集、模型训练、评估和在线服务的全过程中遵循隐私设计原则。在米家应用中, 当智能家居指令理解功能首次启动时, 会显示隐私协议弹窗。

该功能仅在获得明确的用户同意后启用，用户可随时通过设置页面选择退出。在合成训练与评估数据集时，所有标识符字段（例如，用户ID、设备ID）均被严格匿名化。潜在的隐私相关文本（例如，设备别名）会在训练和评估前，通过规范化处理并替换为抽象占位符的方式进行去标识化。最后，我们将数据使用限制在模型开发和质量评估所必需的范围内，并限制在标准安全控制下仅授权人员可访问，来执行数据最小化和治理。

D 局限性

当前MiCU系统的一个主要局限在于其依赖于单轮指令理解，未能纳入长期用户交互历史。智能家居环境本质上是个人化的；然而，我们的模型将每个请求视为独立事件。这种历史上下文的缺失，使得系统无法学习个体用户习惯或“个性”，例如用户一天中不同时段对“温馨”照明的具体偏好。因此，该模型可能难以解决那些需要个性化知识的极端歧义，我们将其确定为未来研究的一个关键方向。